Classical Database Application Development Methodology[#]

Leo Mark, Sham Navathe, & Rocky Dunlap, 2004
revised 2026[*]

Area of Application

This methodology has been created to support the development of medium to large size data intensive applications.

An application is *data intensive* when it needs to maintain and access large amounts of data and therefore will cause a substantial amount of input-output activity (I/O) between computer memory and disk. In contrast, a *process intensive* application is one in which a substantial amount of CPU processing takes place. There are applications that are both data and process intensive. Database management systems were developed to support data intensive applications. Database management systems may not be very useful for a process intensive application. The methodology presented here assumes that an application developed with it will be supported by a relational database management system.

There is a substantial amount of overhead involved in using the methodology. This overhead is probably only justified in *medium* to *large* size application development projects where the chance of failure and the cost of failure are often higher than in *small* projects. In small application development projects all of the following are normally true: the project is well-defined, the development time is short, there is no long-term maintenance, only few people are involved, turnover is small, and, finally, there are no critical resources. Situations where one or more of these are not true lead to a medium to large size project.

Some Basic Assumptions

In order for the methodology to work, we make a couple of assumptions. First, we assume that the business process is known, the documents which are input to and output from the application are known, the tasks of the application are known, by means of scenario descriptions, and the application system boundary is known. If

---

[#] *This methodology is based on several concepts originally presented in N.Roussopoulos,and R.Yeh, "An Adaptable Methodology for Database Design," IEEE Computer, Vol.17, No.5, pp.64-80, May 1984.*

[*] *Minor grammatical errors were corrected, references updated from 4th to 7th edition of Elmasri and Navathe, and styles adjusted to improve document readability/accessibility. Graening, March 2026.*

these assumptions do not hold, then additional work to develop these items is needed before the methodology can be used. Second, we assume that a single database schema unifying all views on data can be designed and agreed upon by all the parties in the organization developing the application. The reason we make this assumption is that a database must have a single schema in order to work. Designing and agreeing on a single database schema is often very difficult. If a single schema cannot be agreed upon, it is our position that the parties developing the application are not collaborating to pursue the goals of the organization they work for.

Database Application Development Methodology

A software development methodology must give guidelines for all its work processes. The following *work processes* are found in most software development methodologies:

- Business process (re-)design
- Analysis
- Specification
- Design
- Implementation
- Testing

We will present guidelines - specifically aimed at database application development - for the work processes: Analysis, Specification, Design, and Implementation. As mentioned above, we assume that the business process, the input and output documents, the tasks, and the application system boundary are known, and we will not present guidelines for Business process (re-)design. Also, we refer to guidelines for testing application software in general, rather than presenting testing guidelines specific to database application development. Finally, we are not discussing organizational and management aspects of database application development. The database application development methodology presented here is an example of a classical waterfall methodology, under which each work process is completed before the next one starts.

The guidelines described for each one of the work processes, Analysis, Specification, Design, and Implementation, include the following:

- Purpose: what we do
- Input: what we start with
- Output: what we end with
- Tool: what we use
- Technique: how we use it

An overview of the methodology is given in Figure 1. Each circle corresponds to a work process as follows:

1. Analysis
2. Specification
3. Design
4. Implementation

Each box corresponds to a model of the system. The result of Analysis is an Information Flow Diagram (IFD), which identifies the major external input and output documents, the major tasks, and the database of the future database application system. It also identifies the system boundary and the potential information flow between the documents, tasks, and the database. The Specification work process has two parts. First, the input and output documents identified in the IFD are used to develop a specification of the database in terms of an Entity-Relationship (ER) or an Extended Entity – Relationship (EER) Diagram[1]. Second, the tasks of the future system are specified in a set of Task Forms. The Task Forms specify what the tasks do relative to the external input and output documents and the database as described by the ER Diagram. The Design work process also has two parts. First, we map the ER Diagram into a relational database schema. Second, we replace the parts of the Task Forms, which specify database queries and data management relative to the ER Diagram by SQL statements expressed against the relational database schema that resulted from the mapping of the ER Diagram. In the Implementation work process, we first implement the relational database schema using SQL on the relational platform, which will be used to support the application system. Then we use the 3rd generation host language such as Java, Visual Basic, C or C++ (including user interface details) to implement the external input and output documents and the control statements around the SQL statements from the Design work process.

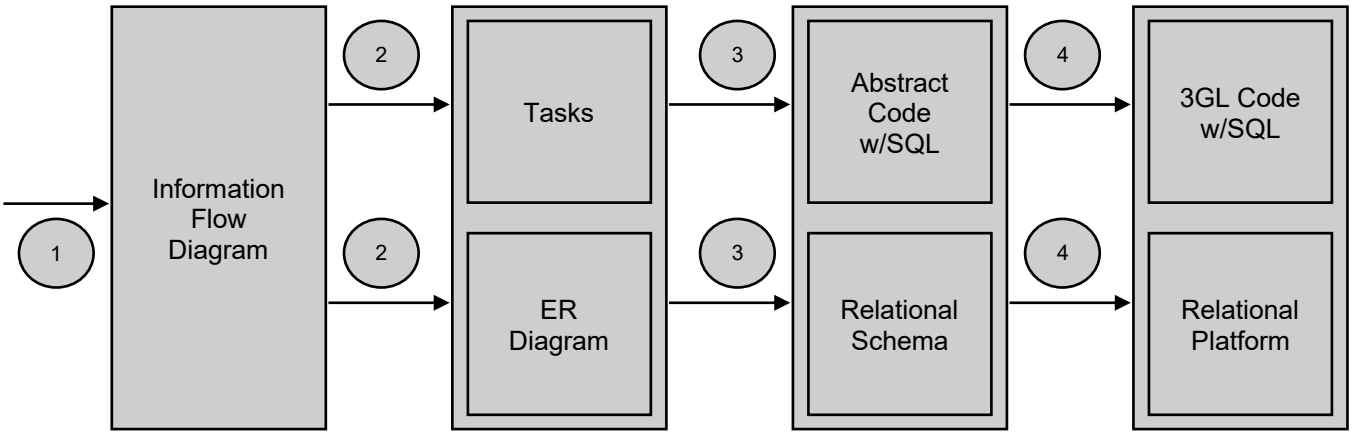


Figure 1. Database Application Development Methodology

[1] Henceforth we use the term ER-Model broadly to include also the Extended Entity-Relationship (EER) Model

An important difference between the database development methodology and other software development methodologies is that it concentrates on data first. Informally, most software development methodologies concentrate on decomposing the tasks into subtasks until the task decomposition is just right. Then, the data are fitted onto the tasks where they belong. In contrast, the database application development methodology concentrates on data first and then fits the tasks onto the data where they belong. Note that Figure 3.1 in Elmasri and Navathe (2016) already points out this two-pronged approach to database design with different emphases.

In the following sections, we will give guidelines for each one of the work processes of database application development methodology.

Analysis

PURPOSE	*analyze documents, tasks, scenarios, etc. to determine the system requirements*
INPUT	*descriptions of documents, tasks, scenarios, usage statistics, plans for the future system, relevant laws, constraints, and policies*
OUTPUT	*Information Flow Diagram (IFD) modeling external documents, tasks, database, information flow, and system boundary. In addition, all information which was use as input to the Analysis work process is carried forward to the specification work process.*
TECHNIQUES	*analysis of documents, scenarios, tasks. Interviews with affected people at all levels of the organization. Reviews of short and long-term plans, manuals, files, and forms. Work from outside in abstraction*
TOOLS	*Information Flow Diagram notation*

An Information Flow Diagram identifies the external input and/or output documents, the tasks, and the database of the future database application system. It also identifies the system boundary and the potential information flow between the documents, tasks, and database. The system boundary is shown as a dotted line. Named boxes outside the system boundary show documents. Tasks are shown by named ellipses inside the system boundary. The database is shown as a box inside the system boundary. In contrast to a Control Flow Diagram, an Information Flow Diagram does not show control flow, it only shows information flow. Information flows from a task to the database if the task inserts, deletes, or updates data in the database. Information flows from the database to a task if the task queries the database. Information flows from a task to an external document if the task writes data on that document, i.e. the document is an output document for that task. Information flows from an external document to a task if the task reads data from that document, i.e. the document is an input document to the task. A document may be both an input and an output document. This is the case, e.g. when the same document structure is used to specify the data used as input for a

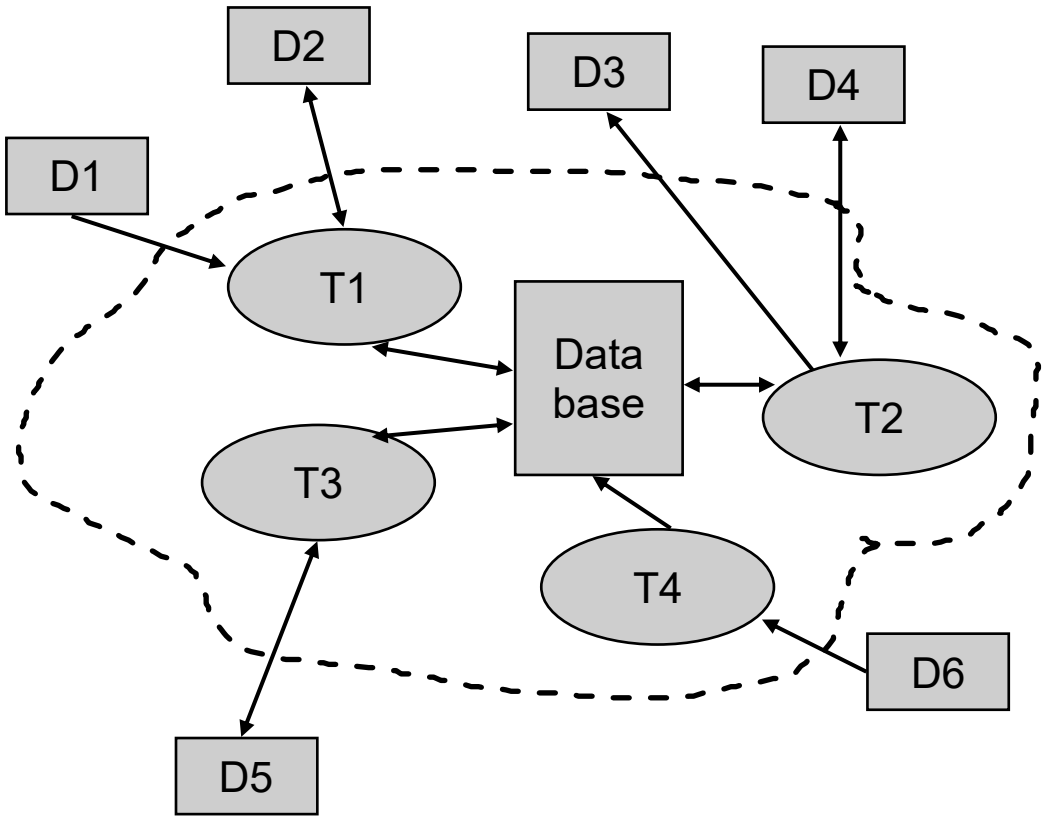


Figure 2. Information Flow Diagram

query and the data that results as output from that query. Documents, which merely control the flow of computation, e.g. menus, documents with buttons, are not shown in the Information Flow Diagram.
A task may have several documents as input and/or output. A document may serve as input and/or output for several tasks. Two tasks are never connected in an Information Flow Diagram. The reason is that the Information Flow Diagram is not supposed to show task decomposition, which is a control flow aspect rather than an information flow aspect. Two documents are never connected in an Information Flow Diagram.

The Information Flow Diagram is supposed to show tasks at a consistently high level of abstraction. It is an important, but rather philosophical observation that the tasks in the Information Flow Diagram should model the real world, not the database application system. That is, the tasks are those described through the real-world scenarios. On the other hand, the Information Flow Diagram *is* the first abstract version of the future database application system. It serves as a starting point for task decomposition during Specification.

Specification

PURPOSE *create a detailed specification of the database and the tasks in the IFD*

INPUT	*IFD, external input and output documents, task scenarios, usage statistics, and other information gathered during the analysis*
OUTPUT	*ER-Diagram, Data Representation, Constraints, Task Decomposition Diagram, and Task Forms*
TECHNIQUES	*data modeling. Top-down decomposition of tasks until their specifications are sufficiently detailed to allow a programmer to implement them from their specifications. Task decomposition may result in multiple tasks replacing the original task or in subtasks controlled by the original task.*
TOOLS	*ER-Model; Task Decomposition Diagram; Task Forms*

Specification is the first work process where; in keeping with the nature of a database application development methodology - we concentrate on data first. We have assumed that the external documents are given. We will now use these documents to specify the database through the use of an ER-Diagram (For a detailed description of the ER-Model and the Extensions, consult Chapter 3 and Sections 4.1 through 4.4 in Elmasri and Navathe, 2016). We do this by asking two basic questions, as follows:

What goes into the database?

- Everything in the database must come from somewhere
- Everything on the input documents must go somewhere

What comes out of the database?

- Everything in the database must be used for something
- Everything on the output documents must come from somewhere

By considering each input document in turn, we identify the entity types and relationship types which must be defined in the database in order to store the data which is inserted, deleted, and/or updated through the input document. By considering the entity types and relationship types in the database one at a time, we guarantee that there is an input document from which that data originates. Each entity type and relationship type in the database must be used for something, i.e. there should be a document to which they are output. By looking through the output documents, one at a time, we make sure that all data present on an output document can be computed from the database. There are a few cases where information on an input document does not end up in the database but is instead used to control the tasks. There are also a few cases where information in the database does not appear on an output document. There may be cases where an organization collects data that are currently not being used in any query but will be used in the future. Such future intended use of data also justifies why that data is collected in the database.

When designing the database from the external input and output documents, it is important to use entity type, attribute, and relationship type names which correspond closely to the corresponding names used on the external documents. It

is often the case that the name of an input document will become the name of an entity type or a relationship type and the names of data fields on a document will become attribute names in the ER-Diagram. Such naming conventions help the developers create the system the organization is expecting, and it helps people in the organization better understand the internals of the system they get.

When the Entity-Relationship Diagram has been completed, we need to determine the *data representation* needed for each one of the attributes in the diagram. The data representation includes information, such as how many characters to use for a Last Name, how many digits for a Zip code, how to represent time and dates, etc. This information should be part of the description of the external documents given at the outset.

A number of *constraints* imposed by laws of nature, laws of society, or by policies of the organization may need to be specified in order that they can be enforced in the database application system we are developing. Often, meticulous attention to constraints is essential in order to guarantee high data quality in the database and application system. Constraints can be specified in structured English, pseudo code, or any other convenient notation. The constraint specifications are given relative to the ER-Diagram.

After the specification of the database in terms of an ER-Diagram, we turn our attention to the specification of the tasks, which will run on the database. The scenario descriptions, which helped us identify the tasks in the IFD, will be used again as we specify each task precisely relative to the external input and output documents and the database as described by the ER Diagram. The specification of the tasks is done on Task Forms.

TASK NAME	*Unique task name*
TASK NUMBER	*Unique task number, e.g. 1, 2, 3, etc. Dot-notation is used for subtasks, e.g. 1.1, 1.2, 1.3*
DESCRIPTION	*Brief natural language description of the task*
ENABLING CONDITION	*Description of what enables the task, e.g. information, control, time, …*
FREQUENCY	*Frequency of task; use same unit of measure across tasks, e.g. #times/day*
INPUT	*List of fields from external input documents; List of entities and relationships from ER-Diagram*
OUTPUT	*List of fields from external output documents; List of entities and relationships from ER-Diagram*
OPERATION	*Detailed pseudo-code description of the task with respect to the external documents and the ER-Diagram*
SUBTASKS	*List of subtasks called by this task.*

Figure 3. Task Form

A Task Form contains the entries shown in Figure 3. Each task has a unique Task Name and Task Number to make for an easy reference. We use the dot-notation (Tasknumber.subtasknumber) to refer to subtasks so it is easy to track which tasks are subtasks of which tasks. Each task has a brief easy to understand natural language Description of what the task does. Every task has Enabling Conditions, e.g. information being available, called by another task, triggered by time, etc. It is important to provide a Frequency for each task to later allow a computation of just how often database queries and updates take place. Since we will compute with frequencies, we need to represent them in a uniform unit of measure. The Frequencies will be used to tune the database. Both Input and Output to a task are precisely specified. A listing of all Entity Types, Relationship Types, and Attributes involved as input and/or output constitutes the specification of the task. Notice that external documents as well as the database may be input and/or output from a task. The most comprehensive entry on a Task Form is the Operation field. Here we specify precisely what the task does relative to the database specified as an ER-Diagram and the external input/output documents. Structured English or pseudo-code is used to provide a clear specification. If subtasks are introduced in the Operations specification, then these are listed in the Subtasks entry.

As Task Forms are filled in to specify the tasks of the future database application system, some of the tasks are hierarchically decomposed into smaller and smaller subtasks. That raises two important questions. First, when are we done with task decomposition? And what are the rules for task decomposition? A good, but not very precise answer to the first question is as follows. We are done with task decomposition when we can give the ER-Diagram and the Task Forms to another programmer, and this programmer can proceed with the design and implementation of the system we are specifying without asking us a number of questions along the way. The second question is addressed by the following Rules of Thumb for Task Decomposition.

Rules of Thumb for Task Decomposition

- Different enabling conditions apply to different parts of the task (this may hold back parts of task able to run)
- Different frequencies apply to different parts of the task (this may result in unnecessary costly indexing)
- Different parts of ER-Diagram used by different parts of the task (this may lock too large parts of database causing lock contention)
- Many subtasks controlled by the task (this may lock a database too long causing lock contention)
- Many diversified operations carried out by the task (this may make the task difficult to understand and program)

A task may be decomposed in two different ways. It may be split into subtasks without any remaining need for the original task. If possible, this is very desirable,

as we will discuss shortly. If not possible, we may split the task into subtasks controlled by the original task. An example of the latter is a transfer of money from one bank account to another where the withdrawal from the first account and the deposit into the second must happen in that order and must both happen for the task to leave the database in a consistent state.

Most of the rules of thumb for task decomposition are inspired by database technology. If different enabling conditions apply to different parts of a task, then it would be desirable to split the task into subtasks; that allows enabled subtasks to run without waiting for other not yet enabled subtasks. It is, in general, easier to schedule many smaller tasks than fewer larger tasks. If different frequencies apply to different parts of a task, then it would be desirable to split the task into subtasks within which all parts have the same frequency. Later, when implementing the database, for each base relation we choose a primary file organization and indices to best support the operations with the highest frequencies on that base relation. Since primary file organizations and indices are costly to maintain, we want to make sure we only do so to support the high frequency tasks which need them. If different parts of the ER-Diagram are used by different parts of the task then it would be desirable to split the task into multiple tasks, such that each uses their separate part of the ER-Diagram. If several parts of the database are used by a task, then there is a high chance the task may cause lock contention in a multi-user database application system. If many subtasks are controlled by a task, then it would be desirable to split the task into multiple tasks, each one with fewer subtasks. The reason is that a task with multiple subtasks will generally lock the database too long and cause more lock contention. If many different operations are carried out by a task, then generally it is more difficult to program correctly, and we may want to split the task into smaller subtasks which are easier to understand and specify.

Finally, we produce a Task Decomposition Diagram summarizing the end-result of the task decomposition process.

At the end of the Specification work process, we have produced an ER-Diagram, which together with the Data Representation and the Constraints specify the database of the future database application system. In addition, we have specified the tasks of the future system - relative to the ER-Diagram - in a set of Task Forms.

Design

PURPOSE	*create detailed design of normalized relational database schema; create detailed design of tasks using pseudo code with embedded SQL; identify need for views*
INPUT	*External Documents, ER-Diagram, Data Representation, Constraints, and Task Forms*

OUTPUT	*Relational Schema Diagram w/primary and foreign keys, constraint definitions in SQL, pseudo code w/SQL, view definitions*
TECHNIQUES	*database normalization; pseudo coding*
TOOLS	*mapping: ER-Model → Relational Model; graphical Data Definition Language (DDL); pseudo code; SQL; views*

Concentrating again on data first, the initial step in the Design work process is to map the ER-Diagram created during the Specification work process to a Relational Database Schema with primary keys and foreign keys. We represent the result of the mapping using a Graphical Data Definition Language (just a fancy way of saying that we construct a diagram of the relational schema and keys resulting from the mapping).

In order to guarantee high data quality, the second step in the Design work process is to make sure the resulting Relational Database Schema is properly normalized. Practical database designers are normally satisfied with relations in Boyce-Codd Normal Form (BCNF), i.e. relations in which the determinants of every functional dependency in a relation is a candidate key of the relation (Chapter 15, Elmasri and Navathe (2016)). It is interesting to notice, that if the ER-Diagram is well designed and explicitly captures *all* the functional dependencies relevant to the application, then the resulting Relational Database Schema is already normalized. Unfortunately, it is often the case that additional functional dependencies are discovered after the ER-Diagram has been completed, in which case some normalization is necessary.

The third step is to design the SQL code to enforce - on the Relational Database - the constraints, which were specified relative to the ER-Diagram during Specification.

We have now designed the Relational Database for the future application system. Our next step is to revise the task forms. The Task Forms specify what the tasks will do relative to the external documents and the database specified by the ER-Diagram. We will revise the Task Forms in the context of the relational schema design to specify what the tasks will do relative to the external documents and the Relational Database. First, we revise the Input and Output entries on the Task Forms. We replace all references to Entity Types, Relationship Types, and Attributes by their counterparts in the Relational Database designed from the ER-Diagram. Then we revise the Operation entry on the Task Forms. The Operation entry essentially contains two kinds of pseudo code, code which deals with the external input and output documents, and code which deals with queries and updates to the database. We now design SQL code which does to the Relational Database what the pseudo code does to the database specified by the ER-Diagram.

After completing the design of the SQL code on all the Task Forms, we analyze the code to determine if any common sub-queries can be factored out. If so, we define Views for each of these sub-queries, and we revise the SQL on the Task Forms to reference the Views.

Implementation

PURPOSE	*implement the conceptual schema; implement the internal schema; implement the pseudo code which interfaces with the Input and Output documents.*
INPUT	*graphical representation of relational schema w/primary and foreign keys, data representation, constraints in SQL, Task Decomposition diagram, Task Forms with pseudo code w/SQL, view definitions.*
OUTPUT	*Implementation of conceptual schema, internal schema, and external schema (the views). Host language implementation of the interface with the external Input and Output documents. Embedding of the SQL queries and updates into the host language program, e.g. as embedded SQL in C, or through JDBC in Java, or similar other means.*
TOOLS	*Relational Database Management System; Host Language; Pre-compilers and Compilers; Logical Access Path (LAP) for Internal Schema implementation.*

Continuing to focus on data first, we now implement the conceptual schema in the Relational Database Management System which will support the application system. This step is straightforward. Given the drawing of the relational schema, the primary and foreign keys, and the data representation, we simply write the `CREATE TABLE` statements in SQL which implement the database using the primitive attribute types provided by the system.

The SQL code implementing the Integrity Constraints was already designed in the Design work process and should be close to complete.

The last, and a very important step in the implementation of the database is the physical layout of each table, or the Internal Schema, as it is normally referred to in the ANSI/SPARC 3-Level Database Architecture. If we were to omit this step and instead complete the implementation of the tasks, then the tasks would actually run correctly on the database (assuming we program everything without errors in syntax and semantics ☺). The only problem is that the tasks may run so slow that the system would be useless in practical situations. In order to understand how to implement the physical layout of the relations, think of the relations as the resources used by all the tasks running on the database. Some relations will be accessed very frequently by several tasks while some will be accessed rather infrequently. Some relations will mostly be queried and will be relatively stable while other relations will be updated frequently and will be highly dynamic. In order to layout the physical structure of the relations we will use the following guidelines

(see part 7 in Elmasri and Navathe (2016) for a discussion of primary file organizations and secondary access via indexes):

- *Primary file organization and indexing* (clustering) of a relation are chosen to best support the task with the highest frequency on that relation.
- *Secondary indices* (non-clustering) are introduced on a relation if:
  - there is a relatively high probability for queries on the base relation
  - the queries are not supported by the primary file organization and indices already chosen, and
  - there is a relatively low probability for updates of the base relation

A convenient way to get an overview of which task is the heaviest user of a relation, we draw a so-called *Logical Access Path Schema*. It consists of a combination of three things:

1. the Task Decomposition Diagram annotated with
2. the SQL statements associated with each task and subtask, and
3. the frequency of each task and subtask

The final step is to use the host language to implement the pseudo code, which deals with the external input and output documents, i.e. the User Interface. Since this is not really different from implementing User Interfaces for other types of applications, we shall not dwell on it in any more detail. The reader is encouraged to study the different approaches to embedding SQL in host languages, e.g. C or C++ with Embedded SQL or JAVA with SQL through JDBC. (see Chapter 10 in Elmasri and Navathe (2016))

A final word of caution

Many database products come with vendor-specific proprietary interface generators and 4th generation languages which make the implementation of the User Interface and the access to the database via SQL very easy to implement. But remember, that if you use these, *then you are confined with that database product for life*. Therefore, it is wiser to implement the application with more general-purpose technology.